# SMART HOME AUTOMATION

*IoT-Based Lighting & Automated Door Control System*

## PROJECT REPORT

Presented By

**PRAKASH RANJAN**

Institute

**National Institute of Technology (NIT), Patna**

# INTRODUCTION

The rapid growth of microcontroller technology and wireless communication has opened new frontiers in residential automation. Smart home systems offer unmatched convenience, energy efficiency, and security by integrating everyday appliances and fixtures into an interconnected digital ecosystem. This project presents an integrated Smart Home Automation system that leverages low-cost, widely available hardware to perform two main functions: IoT-based lighting control and an automated door system.

## Problem Statement

Traditional home systems rely entirely on manual operation. Switching lights on or off requires you to physically interact with the switches on wall panels, and opening doors demands human presence. These conventional approaches suffer from several shortcomings:

- Inefficiency: Lights are frequently left on in unoccupied rooms, resulting in unnecessary energy consumption.
- Inconvenience: Users must physically move to control lighting or operate doors, which is impractical for differently abled individuals or during adverse conditions.
- Lack of Remote Control: Occupants have no mechanism to monitor or manage home systems while away from the premises.
- Security Risks: Manual door operation requires occupants to be physically present, creating potential safety vulnerabilities.

## Objectives

The primary objectives of this project are:

- To design and implement a remote lighting control system utilising the ESP32 microcontroller's Wi-Fi feature and the Blynk IoT platform.
- To develop an automated door access system that uses an ultrasonic sensor to detect human presence and actuate a servo motor.
- To integrate both subsystems into a cohesive smart home solution using readily available, low-cost components.
- To confirm the reliability and stability of sensor-based detection through software-level noise filtering techniques.
- To demonstrate the practicality of building functional smart home automation using open-source hardware and software.

## Scope of the Project

This project is scoped to two primary subsystems, each built around a distinct microcontroller platform:

- IoT Lighting System: An ESP32 development board connects via Wi-Fi to the Blynk mobile application, allowing users to remotely adjust the colour and intensity of an RGB LED module.
- Automated Door System: An Arduino Uno microcontroller processes signals from an ultrasonic distance sensor to detect approaching persons and commands a servo motor to open and close a door mechanism automatically.

# ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project supervisor for their invaluable guidance, constructive feedback, and continuous encouragement throughout the course of this project. Their expertise and insights were instrumental in shaping the direction and quality of this work.

I am also thankful to the faculty and staff of the Department at the National Institute of Technology (NIT), Patna, for providing the laboratory resources, technical support, and a conducive environment for carrying out this project.

Furthermore, I extend my heartfelt thanks to the Ediglobe support team. Their technical expertise, efficiency, and prompt assistance were instrumental in navigating the complex technical challenges encountered during this work's development. Their dedication and willingness to provide timely solutions significantly contributed to the project's success.

Finally, I am deeply grateful to my family and friends for their unwavering support, patience, and motivation, which have been a constant source of strength throughout the duration of this project.

# ABSTRACT

This project presents the design and implementation of an integrated Smart Home Automation system comprising two complementary subsystems: an IoT-based lighting control module and an automated door system. The lighting subsystem employs an ESP32 microcontroller interfaced with an RGB LED module and the Blynk IoT platform, enabling users to remotely adjust room lighting colour and intensity by a smartphone application over Wi-Fi. The automated door subsystem utilises an Arduino Uno microcontroller along with a HC-SR04 ultrasonic distance sensor to detect approaching individuals and actuate a servo motor for door opening, with a software-level multi-reading filter implemented to eliminate false triggers caused by sensor noise.

Both subsystems were independently developed, tested, and validated, demonstrating reliable real-time performance with affordable and widely available hardware. The findings indicate that smart home functionalities typically linked to costly proprietary systems can be effectively replicated using open-source platforms and hobbyist-grade components. This project demonstrates the practical feasibility of such systems and identifies several directions for future development, including expanded appliance control, voice integration, biometric security, and multi-room scalability.

# LIST OF ABBREVIATIONS

1. IoT – Internet of Things
2. ESP32 – Espressif Systems 32-bit Microcontroller
3. RGB – Red, Green, Blue (colour model)
4. PWM – Pulse Width Modulation
5. HC-SR04 – Ultrasonic Distance Sensor Module

# TABLE OF CONTENTS

# 1. LITERATURE REVIEW (SYSTEM OVERVIEW)

The domain of home automation has witnessed significant research activity over the past decade. Prior works have explored diverse communication protocols (Zigbee, Z-Wave, Wi-Fi, Bluetooth), a wide range of microcontrollers (Arduino, Raspberry Pi, ESP8266, ESP32), and various actuation mechanisms. The following subsections contextualise the design choices made in this project relative to the existing body of knowledge.

## 1.1 IoT-Based Lighting Control

Smart lighting has evolved from simple on/off toggle mechanisms to full RGB colour control with scene management. The ESP32, introduced by Espressif Systems, represents a significant upgrade over its predecessor (ESP8266) by incorporating dual-core processing, Bluetooth support, and enhanced GPIO capabilities. For IoT application development, the Blynk platform has emerged as a popular choice owing to its drag-and-drop dashboard, extensive widget library, and straightforward integration with Arduino-compatible microcontrollers.

RGB LED modules allow fine-grained control of three independent colour channels (Red, Green, Blue), enabling millions of colour combinations. By interfacing these modules with pulse-width modulation (PWM) outputs of the ESP32 and controlling them via the Blynk zeRGBa widget, users can create customised ambient lighting scenes suited to different activities and preferences.

## 1.2 Automated Door Systems

Automated door systems have been widely implemented in commercial settings using passive infrared (PIR) sensors, pressure mats, and ultrasonic sensors. For residential applications, ultrasonic sensors (such as the HC-SR04) offer a cost-effective solution, providing reliable distance measurement through the emission and reception of high-frequency ultrasonic sound waves. The HC-SR04 operates on the principle of time-of-flight measurement, calculating object distance based on the elapsed time between an ultrasonic burst and its echo.

Servo motors are widely used in such applications due to their precise angular control. In door automation, servo motors can be mechanically linked to door hinges to effect smooth, controllable opening and closing motions. The Arduino Uno, with its robust ecosystem and ease of programming, serves as an ideal microcontroller for integrating sensor inputs with motor outputs.

## 1.3 System Architecture Overview

The present system is architected around two independent but complementary subsystems. The first subsystem leverages the ESP32's Wi-Fi capability to establish a cloud-mediated control channel between the physical RGB lighting module and a smartphone application. The second subsystem operates as a standalone embedded system, with the Arduino Uno processing local sensor data to drive mechanical actuation without requiring external network connectivity. This architectural separation ensures that a failure in one subsystem does not affect the operation of the other.

# 2. SYSTEM DESIGN AND METHODOLOGY

## 2.1 Hardware Architecture

### 2.1.1 Microcontrollers

Two microcontrollers form the computational backbone of the system. Their complementary capabilities make them well-suited for their respective roles:

| Component | Role | Key Feature Used |
|---|---|---|
| **ESP32 Dev Board** | IoT lighting control | Built-in Wi-Fi, PWM GPIO |
| **Arduino Uno** | Automated door control | Digital I/O, PWM for servo |

### 2.1.2 Sensors and Actuators

- **Ultrasonic Sensor (HC-SR04):** Emits ultrasonic pulses and measures the time taken for the echo to return, computing the distance to the nearest object. It interfaces with the Arduino Uno's digital GPIO pins via Trigger and Echo signals.
- **Servo Motor:** Provides precise angular displacement under PWM control. Mechanically coupled to door hinges, the servo translates electrical signals into rotational motion, physically opening and closing the door.
- **RGB LED Module:** Contains three independent LEDs (red, green, blue), each connected to a dedicated GPIO pin on the ESP32 through protective 330-ohm resistors. PWM signals control the brightness of each colour channel independently.

### 2.1.3 Support Circuitry

Breadboards and jumper wires provide the physical interconnect between components, allowing a solderless, reconfigurable circuit for prototyping. 330-ohm current-limiting resistors are placed in series with each RGB LED channel to prevent excessive current draw that could damage both the LED module and the ESP32 GPIO pins. This resistor value is calculated based on a 3.3V logic output, a forward voltage of approximately 2V across the LED, and a target operating current of 4 mA per channel.

## 2.2 Software Logic and Architecture

### 2.2.1 IoT Interface – Blynk Integration

The ESP32 firmware is developed using the Arduino IDE with the Blynk library. The connection process involves the following steps:

1. Template Creation: A Blynk template is created on the Blynk Cloud dashboard. The Template ID and device authentication token generated are embedded in the firmware.

2. Wi-Fi Provisioning: The ESP32 firmware includes the home network SSID and password. On boot, the ESP32 connects to the Wi-Fi network and authenticates with the Blynk Cloud server.
3. Datastream Configuration: Three integer datastreams are created in Blynk, one each for the Red, Green, and Blue channels, mapped to virtual pins (V1, V2, V3).
4. Dashboard Setup: A zeRGBa widget on the Blynk mobile dashboard is bound to the three datastreams. Additional widgets provide device health monitoring.
5. Callback Functions: The ESP32 firmware implements BLYNK_WRITE handlers for each virtual pin. When a user adjusts the zeRGBa slider, the handler reads the intensity value (0–255) through Param.asInt() function and writes it to the appropriate GPIO PWM channel.

### 2.2.2 Proximity Detection Logic – Counter Variable Method

A key design consideration in the automated door subsystem is minimizing false positive detections caused by transient noise in ultrasonic sensor readings. The firmware implements a counter variable method with five consecutive confirmations:

- Initial Detection: The sensor takes a distance measurement. If the measured distance falls within the defined detection threshold, a detection counter is incremented.
- Consecutive Confirmation: The system immediately takes four additional readings. Each reading within the threshold increments the counter; any reading outside resets the counter to zero.
- Threshold Validation: Only when the counter reaches five does the system treat the detection as valid and trigger door opening.
- Reset Mechanism: After the door completes its open-wait-close cycle, the counter is reset to zero and the system returns to idle monitoring mode.

# 3. IMPLEMENTATION DETAILS

## 3.1 IoT-Based Lighting System

### 3.1.1 Hardware Assembly

The RGB LED module is connected to the ESP32 development board as follows. Each of the three colour pins (R, G, B) on the module is connected through a 330-ohm resistor to a dedicated PWM-capable GPIO pin on the ESP32. The common cathode of the RGB module is connected to the ground rail of the breadboard, which is in turn connected to the GND pin of the ESP32. The ESP32 is powered via USB from a computer or a 5V adapter.

### 3.1.2 Blynk Application Configuration

The Blynk mobile application dashboard is configured with the following elements for the lighting subsystem:

- zeRGBa Widget: The primary control interface. This widget presents a colour picker area where the user moves a cursor to select a colour. In 'Simple Mode', each channel value is sent to its assigned virtual pin independently.
- Connection Status Widget: Displays the online/offline status of the ESP32 device in real time.
- Last Connection Widget: Logs the timestamp of the most recent connection event for diagnostic purposes.

### 3.1.3 Firmware Logic

The ESP32 firmware initialises the Blynk connection using the Template ID, device name, and authentication token as defined in the Blynk app. In the main loop, Blynk.run() maintains the cloud connection and processes incoming commands. Three BLYNK_WRITE(Vx) callback functions intercept slider events from the zeRGBa widget and apply the received integer value (0–255) as an analog write to the corresponding RGB GPIO pin, dynamically adjusting LED brightness via PWM.

## 3.2 Automated Door System

### 3.2.1 Hardware Assembly

The automated door system hardware is assembled on a breadboard with the following connections:

The HC-SR04 ultrasonic sensor's VCC and GND pins connect to the Arduino's 5V and GND pins, respectively. The Trigger pin connects to a digital output pin (e.g., D9), and the Echo pin connects to a digital input pin (e.g., D10). The servo motor's signal wire connects to a PWM-capable digital pin (e.g., D6) on the Arduino, while its power and ground wires connect to the 5V and GND rails. The servo's horn or wing attachment point is mechanically linked to the door's hinge mechanism.

### 3.2.2 Detection and Actuation Sequence

The complete door operation sequence proceeds as follows:

1. Idle State: The Arduino continuously polls the ultrasonic sensor, sending trigger pulses and measuring echo return times.
2. Object Detection: Upon measuring a distance below the defined threshold, the counter variable is incremented.
3. Confirmation Loop: Four additional readings are taken immediately. If all five consecutive readings confirm presence (counter reaches 5), the system transitions to the active state.
4. Door Opening: The Arduino sends a PWM signal commanding the servo to rotate 60–75 degrees, mechanically opening the door.
5. Hold Timer: A delay timer is activated. The door remains open for 15–30 seconds (configurable via firmware constant).
6. Door Closing: After the delay, the Arduino commands the servo to rotate back to its initial 0-degree position, closing the door.
7. System Reset: The counter variable is reset to zero, and the system returns to the idle polling state.

# 4. RESULTS AND DISCUSSION

## 4.1 System Performance

Both subsystems were tested under controlled conditions to evaluate their functionality and reliability. The key performance outcomes are summarised below.

| Metric | IoT Lighting System | Automated Door System |
|---|---|---|
| Wi-Fi Connectivity | Stable connection via Blynk Cloud | N/A (standalone) |
| Control Latency | < 1 second response on slider change | Near-instantaneous (< 0.5s) |
| Colour Accuracy | Full 0–255 per channel | N/A |
| Detection Accuracy | N/A | High (false positives eliminated) |
| Door Response | N/A | 60–75° rotation on valid detection |
| Auto-Close Delay | N/A | 15–30 seconds (configurable) |
| System Stability | Continuous operation without crashes | Reliable cyclic reset |

## 4.2 IoT Lighting – Observations

The IoT lighting system demonstrated seamless integration between the physical RGB LED module and the Blynk mobile application. Key observations include:

- The ESP32 successfully authenticated with the Blynk Cloud and maintained a stable Wi-Fi connection throughout extended testing sessions.
- Slider adjustments on the zeRGBa widget were reflected in the physical LED output within approximately one second, confirming low-latency cloud-to-device communication.
- All three colour channels (R, G, B) responded independently and proportionally to slider inputs, producing the full spectrum of colours through additive mixing.
- The connection status and last-seen widgets accurately reflected the device's online state, providing useful diagnostic feedback.

## 4.3 Automated Door – Observations

The automated door system performed reliably across multiple test cycles. Key observations include:

- The five-consecutive-reading filter (counter variable method) substantially reduced false positive detections. Single transient echoes did not trigger the door mechanism.
- The servo motor executed the 60–75-degree rotation smoothly, opening the door hinge-linkage without binding or stalling.
- The auto-close functionality operated reliably, with the door returning to the closed position consistently after the configured delay.
- Counter reset after each cycle ensured that the detection system was immediately ready for subsequent person detection without manual intervention.
- During testing, occasional edge cases were noted where very slow approaches near the detection threshold boundary caused brief counter resets, resulting in a slightly delayed door opening. This is expected behaviour given the conservative filtering design.

## 4.4 Challenges Encountered

Several challenges were identified and resolved during implementation:

- Wi-Fi Dropout: Intermittent connectivity caused the Blynk connection to drop during early testing. Resolved by implementing Blynk.connect() with a timeout loop to handle reconnection gracefully.
- Servo Jitter: Initial servo configurations produced minor jitter at the rest position due to noise on the PWM signal. Adding a small capacitor across the servo power supply terminals reduced this effect.
- Ultrasonic Interference: In environments with hard reflective surfaces, multi-path echoes occasionally produced erroneous short-range readings. The five-reading confirmation window effectively masked this interference.

# 5. CONCLUSION AND FUTURE WORK

## 5.1 Conclusion

This project successfully demonstrates the feasibility of designing and deploying a functional smart home automation system using low-cost, widely accessible microcontrollers and open-source software platforms. By combining the ESP32's Wi-Fi capabilities with the Blynk IoT framework, the project delivers intuitive, real-time remote control over room lighting. Simultaneously, the Arduino Uno's deterministic control capabilities are leveraged to implement a reliable, hands-free automated door system driven by ultrasonic proximity sensing.

Both subsystems performed as intended during testing. The IoT lighting system provided seamless mobile-to-hardware control with sub-second latency, while the automated door system demonstrated reliable detection accuracy through its multi-reading confirmation algorithm. The project validates that smart home functionality traditionally associated with expensive proprietary systems can be replicated using affordable, hobbyist-grade hardware with careful firmware design.

The successful integration of hardware components, embedded firmware, and cloud-based IoT services in this prototype provides a solid foundation for more sophisticated home automation systems, contributing to the broader goals of smart living, energy conservation, and enhanced quality of life.

## 5.2 Future Work

Several directions exist for extending and improving the current system:

- Expanded IoT Appliance Control: The existing Blynk infrastructure can be extended to control additional home appliances such as fans, air conditioners, and smart plugs by adding relay modules to the ESP32.
- Voice Control Integration: Incorporating voice assistant compatibility (e.g., Amazon Alexa or Google Assistant) via IFTTT or direct API integration would further enhance accessibility.
- Biometric Door Security: Replacing or augmenting the ultrasonic sensor with a fingerprint scanner or RFID card reader would add a layer of identity verification to the door access system.
- Energy Monitoring: Adding current sensors to the lighting circuit would allow the system to log and report energy consumption data through the Blynk dashboard.
- Multi-Room Scalability: The architecture could be extended to support multiple ESP32 nodes across different rooms, all managed from a central Blynk dashboard.
- PCB Design: Migrating from a breadboard prototype to a custom PCB would improve reliability, reduce noise, and make the system suitable for permanent installation.
- Machine Learning for Presence Detection: Training a simple ML model on sensor data could improve detection accuracy and enable nuanced behaviours such as distinguishing between humans and pets.

# 6. REFERENCES

The following references informed the design, implementation, and analysis presented in this report:


1. Espressif Systems. (2023). ESP32 Technical Reference Manual. https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
2. Blynk Inc. (2024). Blynk Documentation – Getting Started with ESP32. https://docs.blynk.io/en/getting-started/what-do-i-need-to-blynk
3. Arduino. (2024). Arduino Uno Rev3 Datasheet. https://store.arduino.cc/products/arduino-uno-rev3
4. HC-SR04 Ultrasonic Sensor Datasheet. (2022). Cytron Technologies. https://docs.cytron.io/sensors/ultrasonic-sensor/sn-hc-sr04
5. Kumar, A., & Singh, R. (2021). IoT-Based Smart Home Automation Using ESP32 and Blynk Platform. International Journal of Engineering Research and Technology (IJERT), 10(3), 213–218.
6. Miorandi, D., Sicari, S., De Pellegrini, F., & Chlamtac, I. (2012). Internet of things: Vision, applications and research challenges. Ad Hoc Networks, 10(7), 1497–1516.
7. Gunge, V. S., & Yalagi, P. S. (2016). Smart home automation: A literature review. International Journal of Computer Applications, 975, 8887.
8. Servo Motor Basics. (2023). SparkFun Learn. https://learn.sparkfun.com/tutorials/hobby-servo-tutorial

# 7. APPENDICES

## Appendix A: Arduino Code – Automated Door Control System

*File: Arduino_Smart_door.ino*

```
#include <Servo.h>
const int trigPin = 9;
const int echoPin = 6;
const int servoPin = 11;
const int detectionDistance = 15;
const int openAngle = 0;
const int closeAngle = 60;
const int holdDelay = 15000;
const int filterSteps = 5;

Servo doorServo;
// Filter counter variable
int detectionCount = 0;

void setup() {
  doorServo.attach(servoPin); doorServo.write(closeAngle);  pinMode(trigPin, OUTPUT);  pinMode(echoPin,
INPUT);  Serial.begin(9600);
}

void loop() {
  if (ObjectPresent()) {
    Serial.println("Object confirmed! Opening door...");
    doorServo.write(openAngle);
    delay(holdDelay);
    detectionCount = 0;
  }

  else {
    doorServo.write(closeAngle);
  }
}

bool ObjectPresent() {
  long duration;
  int distance;

  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  duration = pulseIn(echoPin, HIGH);
  distance = duration * 0.0343 / 2;

  Serial.print("Current Distance: ");
  Serial.print(distance);
  Serial.print(" cm | Consecutive Count: ");

  if (distance > 0 && distance <= detectionDistance) {
    detectionCount++;
  } else {
    detectionCount = 0;
  }

  Serial.println(detectionCount);
```

```
  if (detectionCount >= filterSteps) {
    return true;
  }

  return false;
}
```

# Appendix B: ESP32 Code – IoT-Based Lighting Control System

*File: ESP32_code_IOT_Lighting.ino*

```
#define BLYNK_TEMPLATE_ID   "TMPL3kOyOaLpy"
#define BLYNK_TEMPLATE_NAME "AMBIENT LIGHTING"
#define BLYNK_AUTH_TOKEN    "MHsSyFNxqOklsFbcOHuDR3OLSpSJ76mq"
#define RED_PIN   15
#define GREEN_PIN 2
#define BLUE_PIN  19

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
char auth[] = BLYNK_AUTH_TOKEN; char ssid[] = "……"; char pass[] = "…….";

void setup() {
  pinMode(RED_PIN, OUTPUT); pinMode(GREEN_PIN, OUTPUT); pinMode(BLUE_PIN, OUTPUT);
  Blynk.begin(auth, ssid, pass);
}

void loop() {
  Blynk.run();
}

BLYNK_WRITE(V0) {
  int r = param.asInt();
  analogWrite(RED_PIN, r);
}

BLYNK_WRITE(V1) {
  int g = param.asInt();
  analogWrite(GREEN_PIN, g);
}

BLYNK_WRITE(V2) {
  int b = param.asInt();
  analogWrite(BLUE_PIN, b);
}
```